

Design Patterns

Expense Tracker Web Application

Group Members

Nandita	Yashoda	Sabina	Sujata
----------------	----------------	---------------	---------------

Project Title	Expense Tracker Web Application
Course	Software Engineering
Document Type	Design Pattern Documentation
Technology	HTML · CSS · JavaScript · localStorage
Total Patterns	8 Design Patterns

Table of Contents

1	MVC Pattern ★ Core Core architecture — Model, View, Controller separation
2	Layered Architecture Organised layers: UI → Logic → Data → Storage
3	Authentication Pattern Secure login, register, and logout functionality
4	Session Management Data persistence across browser sessions via localStorage
5	Repository Pattern Centralised data access over localStorage
6	Factory Pattern Consistent object creation for expenses and budgets
7	Observer Pattern Automatic UI updates when data changes
8	Singleton Pattern Single shared instance of each repository

MVC Pattern

Field	Details
Pattern Name	MVC Pattern
Description	Model-View-Controller (MVC) is the primary architectural pattern of the Expense Tracker. It divides the entire application into three clearly separated layers — Model, View, and Controller — each with a distinct responsibility, ensuring that the user interface, data management, and application logic never become entangled.
Problem Description	In a single-page JavaScript application, HTML rendering, data manipulation, and event handling can easily merge into one unstructured block, making the application fragile. Any change to the UI risks breaking data logic, and adding new features becomes increasingly difficult and error-prone. A clear architectural boundary between presentation, logic, and data is required to keep the Expense Tracker maintainable, testable, and extensible.
Solution Description	The application is divided into three explicit layers: Model — Managed entirely by ExpenseRepository and BudgetRepository. Responsible for all data storage, retrieval, and persistence to localStorage. The Model never knows anything about the UI. View — All rendering functions: renderStats(), renderList(), renderChart(), renderCategoryBreakdown(), renderRecent(), and renderBudgetBar(). Each reads from the Model and produces visible HTML output. The View never writes data. Controller — Event-driven functions: addExpense(), deleteExpense(), openEdit(), setBudget(). These respond to user input, call the Model to update data, and trigger the View to re-render via renderAll().
Consequences	Each layer can be modified independently. The UI can be redesigned without touching data logic. Storage can be replaced without changing rendering code. New features slot cleanly into the appropriate layer. The minor trade-off is slightly more structural overhead, far outweighed by maintainability and clarity.

Layered Architecture

Field	Details
Pattern Name	Layered Architecture
Description	Layered Architecture organises the application into horizontal tiers where each layer has a specific responsibility and communicates only with the layer directly below it. This creates a clean separation of concerns across the entire system.
Problem Description	Without a defined layer structure, the Expense Tracker's UI code, business calculation logic, and data access calls would be mixed together. A change in how data is stored would require searching and modifying code throughout the entire application, increasing the risk of bugs and making the system fragile.
Solution Description	The Expense Tracker is structured into four distinct layers: 1. Presentation Layer — HTML/CSS and all View rendering functions. Solely responsible for displaying data to the user. 2. Controller Layer — JavaScript event handlers and the Controller object. Processes user input and coordinates between Presentation and Business layers. 3. Business Logic Layer — Validation, calculation, and formatting functions such as budget percentage calculations, category totals, and CSV generation. 4. Data Access Layer — ExpenseRepository and BudgetRepository handle all reads and writes to localStorage, completely hiding storage details from upper layers.
Consequences	Each layer can be updated, replaced, or tested without affecting other layers. For example, localStorage could be replaced with a server API by changing only the Data Access Layer. The main trade-off is that all requests pass through every layer, adding minor structural overhead.

Authentication Pattern

Field	Details
Pattern Name	Authentication Pattern
Description	The Authentication Pattern controls access to the application by verifying the identity of users before granting entry. It encompasses the complete login, registration, and logout flow, ensuring that only verified users can view and manage expense data.
Problem Description	Without authentication, anyone who opens the Expense Tracker URL would immediately have full access to all expense records and budget data. In a shared or academic environment this is unacceptable — each user must have their own secure, private access to the application. A login and registration system is required to identify users, protect their data, and control who can enter the application.
Solution Description	The Auth object manages the complete authentication lifecycle. User accounts are stored in localStorage under the key 'et_users' as a JSON array of {username, password, name} records. A demo account is seeded automatically on first load. Login: Auth.login() reads the submitted username and password, searches the users array for a matching record, and grants access if found. An error message is displayed for invalid credentials. Register: Auth.register() validates all fields — non-empty inputs, minimum password length of 6 characters, password confirmation match, and username uniqueness — before creating and storing the new account. Logout: Auth.logout() clears the active session and returns the user to the login page, preventing further access without re-authentication.
Consequences	Only authenticated users can access the expense dashboard, protecting data privacy. New users can self-register without admin intervention. The logout function ensures sessions can be terminated cleanly. The trade-off is that passwords are stored in plain text in localStorage, which is acceptable for an academic project but would require hashing (e.g. bcrypt) in production.

Session Management

Field	Details
Pattern Name	Session Management
Description	Session Management ensures that the authenticated user's identity and application state are preserved across page refreshes and browser restarts. Once a user logs in, their session is maintained automatically so they do not need to re-enter credentials every time they visit the application.
Problem Description	Without session management, every page refresh would log the user out and return them to the login screen, even mid-task. This makes the application frustrating to use and breaks the continuity of the user experience. A persistent session mechanism is required so that authenticated users remain logged in across the application lifecycle until they explicitly sign out.
Solution Description	When a user successfully logs in or registers, <code>Auth.setSession()</code> serialises the user object to JSON and stores it in <code>localStorage</code> under the key <code>'et_session'</code> . On every application load, <code>Auth.init()</code> checks for an existing session using <code>Auth.getSession()</code> . If a valid session is found, the user is taken directly to the dashboard without needing to log in again. When the user clicks Sign Out, <code>Auth.logout()</code> calls <code>Auth.clearSession()</code> , which removes the <code>'et_session'</code> key from <code>localStorage</code> , terminating the session and returning the user to the login page.
Consequences	Users enjoy a seamless, uninterrupted experience — logging in once and staying authenticated across refreshes. Session termination via logout is clean and immediate. The trade-off is that sessions have no expiry time, meaning a session persists indefinitely until the user explicitly logs out or clears browser storage.

Repository Pattern

Field	Details
Pattern Name	Repository Pattern
Description	The Repository Pattern introduces a dedicated layer that centralises all data access logic. It acts as the only point of contact between the application and the underlying data store, exposing a clean collection-style interface and completely hiding the storage mechanism from the rest of the application.
Problem Description	The Expense Tracker requires frequent read and write operations for expenses and budgets across multiple parts of the codebase. Without this pattern, localStorage calls would be scattered throughout event handlers, UI functions, and calculation logic — making the code difficult to maintain, test, or modify. This pattern is needed wherever data operations must be centralised and the application logic must not depend on how or where data is physically stored.
Solution Description	Two repository objects are defined as Immediately Invoked Function Expressions (IIFEs), creating singleton instances: ExpenseRepository and BudgetRepository. Each exposes standard methods — getAll(), add(), update(), delete() — that the Controller calls exclusively. Internally, each repository handles JSON serialisation and deserialisation to and from localStorage. No other part of the application ever accesses localStorage directly.
Consequences	The UI and business logic remain fully decoupled from the storage implementation. Replacing localStorage with a REST API or IndexedDB in future requires changes only within the repository layer. Unit testing is simplified as repositories can be mocked independently. The minor trade-off is an extra layer of indirection, far offset by significantly improved maintainability.

Factory Pattern

Field	Details
Pattern Name	Factory Pattern
Description	The Factory Pattern centralises object creation through dedicated factory functions. Instead of constructing data objects inline at multiple locations, all parts of the application call a single factory that guarantees every created object has a consistent, complete, and valid structure.
Problem Description	In the Expense Tracker, new expense objects are created when a user submits the form, when data is imported from CSV, and when sample data is loaded. If each location builds the object differently — missing a field, using an inconsistent date format, or incorrect data type — the inconsistencies cause bugs in chart rendering, budget calculations, and CSV export. A Factory is needed wherever the same type of structured object must be created in multiple contexts with a guaranteed, uniform shape.
Solution Description	Two factory objects are defined: ExpenseFactory and BudgetFactory. ExpenseFactory.create(title, amount, category, date) generates a unique ID using Date.now(), trims whitespace from the title, parses the amount to a two-decimal float, and returns a complete, well-formed expense object. BudgetFactory.create(amount) ensures the budget object always has the correct structure. All parts of the application call these factories exclusively rather than constructing objects manually.
Consequences	Every expense and budget object in the system is guaranteed to have the correct fields, data types, and format. Adding a new required field requires updating only the factory function, and the change propagates automatically to all creation points. The minor trade-off is that developers must consistently use the factory rather than creating objects inline.

Observer Pattern

Field	Details
Pattern Name	Observer Pattern
Description	The Observer Pattern separates the display of object state from the object itself. It allows multiple independent UI components to automatically react and update whenever the underlying data changes, without requiring each component to be manually notified after every operation.
Problem Description	The Expense Tracker has multiple UI components — the expense list, the budget progress bar, the Chart.js doughnut chart, the category breakdown table, the statistics cards, and the recent transactions list — all depending on the same data. Without this pattern, every Controller function must manually call each UI update function, which is error-prone and leads to missed updates and inconsistent display states across the application.
Solution Description	A central <code>renderAll()</code> function acts as the subject. Whenever data changes through any Controller operation — <code>addExpense()</code> , <code>deleteExpense()</code> , <code>saveEdit()</code> , or <code>setBudget()</code> — the Controller calls <code>renderAll()</code> . This function then notifies and triggers all registered observer components in sequence: <code>renderStats()</code> , <code>renderBudgetBar()</code> , <code>renderList()</code> , <code>renderChart()</code> , <code>renderCategoryBreakdown()</code> , and <code>renderRecent()</code> . Each observer independently re-reads the latest data from the repository and re-renders its section of the UI.
Consequences	All UI components remain consistently in sync with the application data at all times. Adding a new display section requires only registering a new render function inside <code>renderAll()</code> . The trade-off is that all observers re-render on every change, even when unrelated to their data, which may cause minor redundant processing at scale.

Singleton Pattern

Field	Details
Pattern Name	Singleton Pattern
Description	The Singleton Pattern ensures that a class or object is instantiated only once throughout the entire application lifecycle and provides a single global point of access to that instance. It prevents duplicate instances that could cause conflicting state in the application.
Problem Description	In the Expense Tracker, both ExpenseRepository and BudgetRepository manage shared application state stored in localStorage. If multiple instances of either repository were created at different points, each could independently cache or write different data, resulting in data conflicts, corruption, or inconsistent reads across different UI components. This pattern is required wherever exactly one object must coordinate all access to a shared resource such as a storage layer or configuration manager.
Solution Description	Both ExpenseRepository and BudgetRepository are implemented as JavaScript Immediately Invoked Function Expressions (IIFEs) that execute once at application startup and return a single, fixed object literal. The IIFE pattern guarantees the initialisation logic runs exactly once and the returned object is the only instance ever accessible. All Controller functions and View components reference this same object, ensuring all reads and writes are coordinated through one unified access point.
Consequences	Data consistency is guaranteed across all UI components and controller functions since they share the same repository instance. Memory usage is minimised by avoiding redundant object creation. The trade-off is that the singleton holds global state, which can make isolated unit testing more complex, requiring explicit clearing of localStorage between test cases.

Thank You

for reviewing our project

This document presents the eight design patterns applied in the development of our Expense Tracker web application. We hope this documentation clearly demonstrates our understanding of software design principles and clean architecture.

Nandita

Yashoda

Sabina

Sujata

Expense Tracker Project